

# UML Submission Report

## Table of Contents

- [API Code Integration for NAMASTE & ICD-11 TM2 - "CodeVeda"](#)
- [Platform Overview](#)
- [UML Concepts and Methodology](#)
- [UML Diagrams for CodeVeda Platform](#)
- [Conclusion](#)
- [Technical Specifications](#)
- [Future Enhancements](#)

### API Code Integration for NAMASTE & ICD-11 TM2 - "CodeVeda"

#### **Executive Summary**

**CodeVeda** is a FHIR-compliant API-based microservice designed to bridge the interoperability gap between traditional AYUSH systems (NAMASTE) and modern biomedicine (WHO ICD-11 TM2). The system leverages a smart mapping engine to analyze diagnostic queries and automatically translate traditional Ayurvedic terms into globally recognized ICD-11 codes, enabling real-time dual coding.

The platform serves as a comprehensive integration solution for healthcare providers and EMR systems seeking standardized clinical documentation, featuring secure ABHA authentication, automated FHIR bundle generation, and strict adherence to India's EHR standards to streamline insurance claims and clinical analytics.

#### **Platform Overview**

##### **Key Features :**

**Real-time Terminology Integration:** Seamless synchronization of internal NAMASTE datasets with the external WHO ICD-11 API.

**Smart Dual Coding Mechanism:** Automatic mapping algorithms that provide instant ICD-11 TM2 equivalent suggestions for traditional AYUSH diagnoses.

**FHIR Bundle Generation:** One-click conversion of dual-coded medical encounters into standardized Health Level Seven (HL7) FHIR R4 resources.

**User Management & Security:** Native ABHA authentication utilizing OAuth 2.0 and JWT tokens for verified practitioner access.

**Intelligent Audit System:** Automated activity logger that records every user search, mapping confirmation, and system interaction for strict compliance.

## **System Architecture**

**CodeVeda** follows a modular architecture comprising:

- Frontend Layer:** React-based web dashboard providing unified interfaces for terminology lookup and dual coding.
- Authentication Layer:** ABHA-integrated OAuth 2.0 service for secure user management and access control.
- API Management Layer:** RESTful endpoints handling internal lookups, external data source integration, and rate limiting.
- Mapping & FHIR Service Layer:** Core logic responsible for cross-referencing terminologies and generating standard FHIR bundles.
- Logging Service Layer:** Dedicated Activity Logger service for immutable audit trails and non-repudiation tracking.
- Database Layer:** Dual-database implementation utilizing MongoDB for unstructured terms and PostgreSQL for robust transactional logs.

## **UML Concepts and Methodology**

### **Unified Modeling Language (UML) Overview**

The Unified Modeling Language (UML) is a standardized modeling language used in software engineering to visualize, specify, construct, and document software systems. UML provides a comprehensive set of diagrams that represent different aspects of a system from various perspectives.

### **UML Diagram Categories**

#### **1. Structural Diagrams**

Define the static structure of the system

Show the organization of system components

Include: Class, Object, Component, Deployment, Package, and Profile diagrams

#### **2. Behavioral Diagrams**

Describe the dynamic behavior of the system

Show interactions between system components over time

Include: Use Case, Activity, State Machine, and Interaction diagrams

#### **3. Interaction Diagrams (Subset of Behavioral)**

Focus on the flow of control and data among system components

Include: Sequence, Communication, Timing, and Interaction Overview diagrams

# UML Diagrams for Codeveda Platform

## 1. Use Case Diagram

**Purpose:** Defines the functional requirements and interactions between external actors and the CodeVeda system.

**Key Elements:**

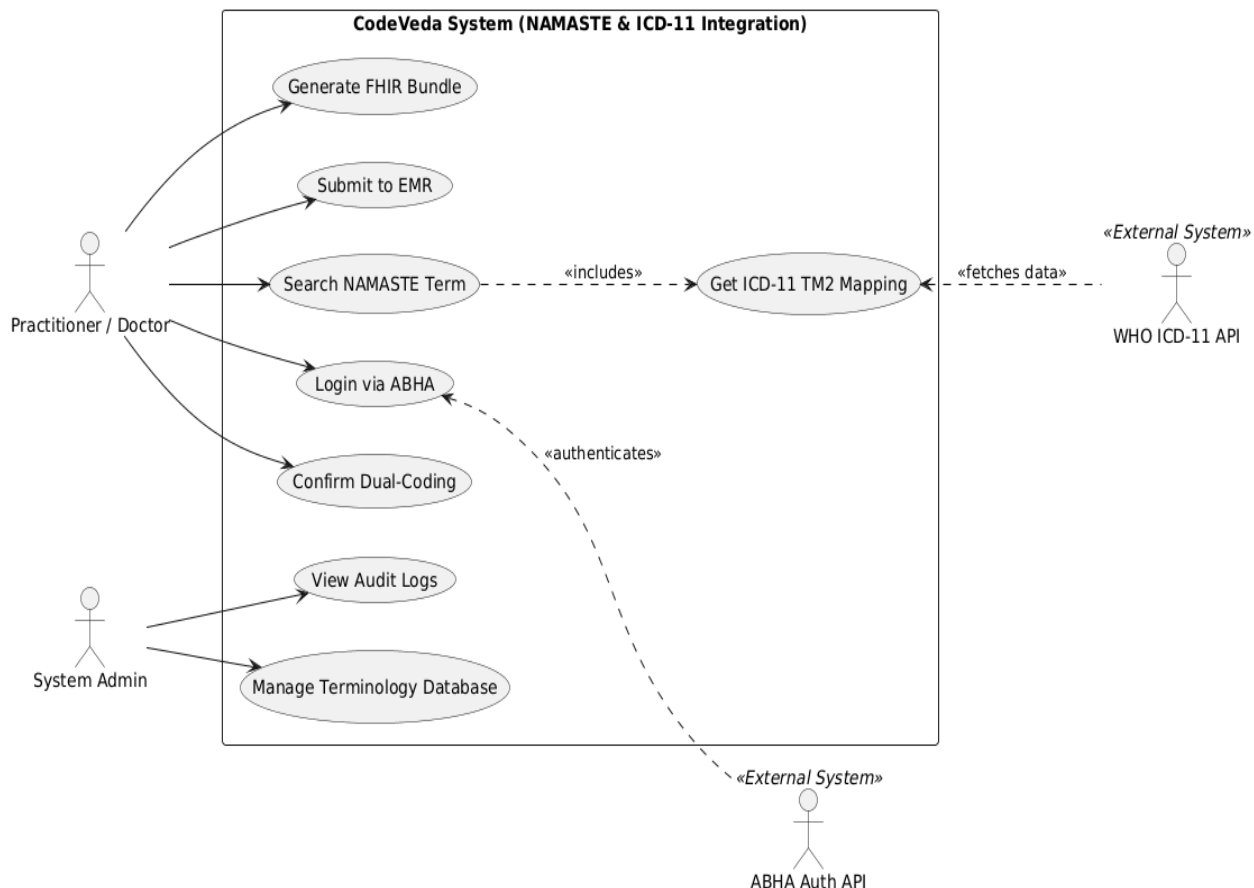
**Actors:** Practitioner / Doctor, System Admin, External System (ABHA Auth API), External System (WHO ICD-11 API).

**Use Cases:** Login via ABHA, Search NAMASTE Term, Get ICD-11 TM2 Mapping, Confirm Dual-Coding, Generate FHIR Bundle, Submit to EMR, View Audit Logs, Manage Terminology Database.

**Relationships:** Association (actors connecting to use cases), inclusion (<<includes>> from Search to Mapping), and system dependencies (<<authenticates>> from ABHA API, <<fetches data>> from WHO API).

**Description:** It highlights the primary workflow of a medical practitioner who logs in securely via the ABHA network, searches for traditional NAMASTE terms, and receives automated ICD-11 TM2 mappings fetched directly from the WHO API.

It further demonstrates how the system finalizes this dual-coded data by generating standard FHIR bundles for EMR submission



## Use Case Diagram

## 2. Class Diagram

**Purpose:** Displays the static structure of the CodeVeda system, including its core classes, attributes, methods, and their interdependencies.

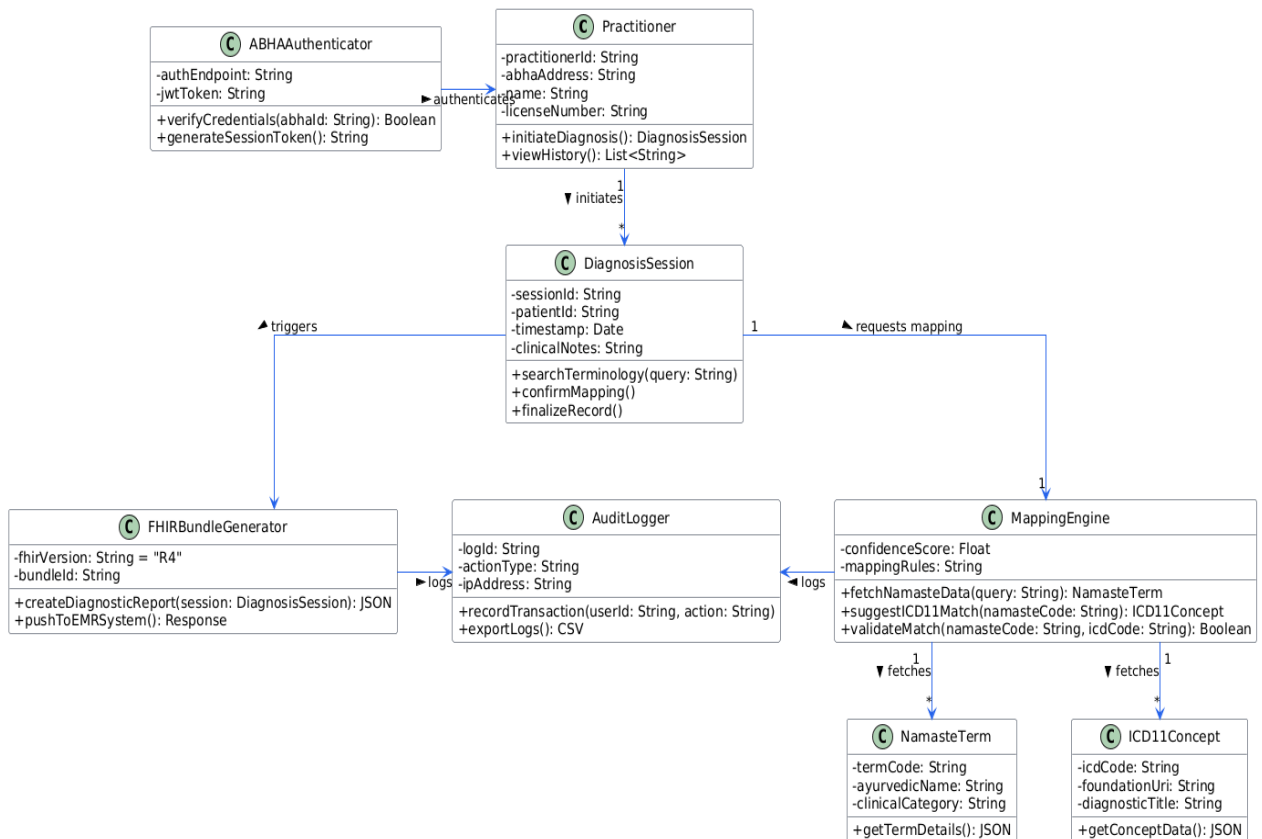
### Key Elements:

**Classes:** Practitioner, ABHAAuthenticator, DiagnosisSession, MappingEngine, NamasteTerm, ICD11Concept, FHIRBundleGenerator, AuditLogger.

**Attributes:** Authentication tokens, session details, confidence scores, and standardized code formats.

**Relationships:** Shows a clear top-down hierarchical flow of the backend architecture.

**Description:** This diagram maps out the complete workflow where an ABHAAuthenticator verifies a Practitioner, who then initiates a DiagnosisSession. This session interacts with a MappingEngine to fetch both NamasteTerm and ICD11Concept data. Finally, the system triggers the FHIRBundleGenerator to create standardized records, while the AuditLogger records all critical backend transactions.



## Class Diagram

### 3. Sequence Diagram

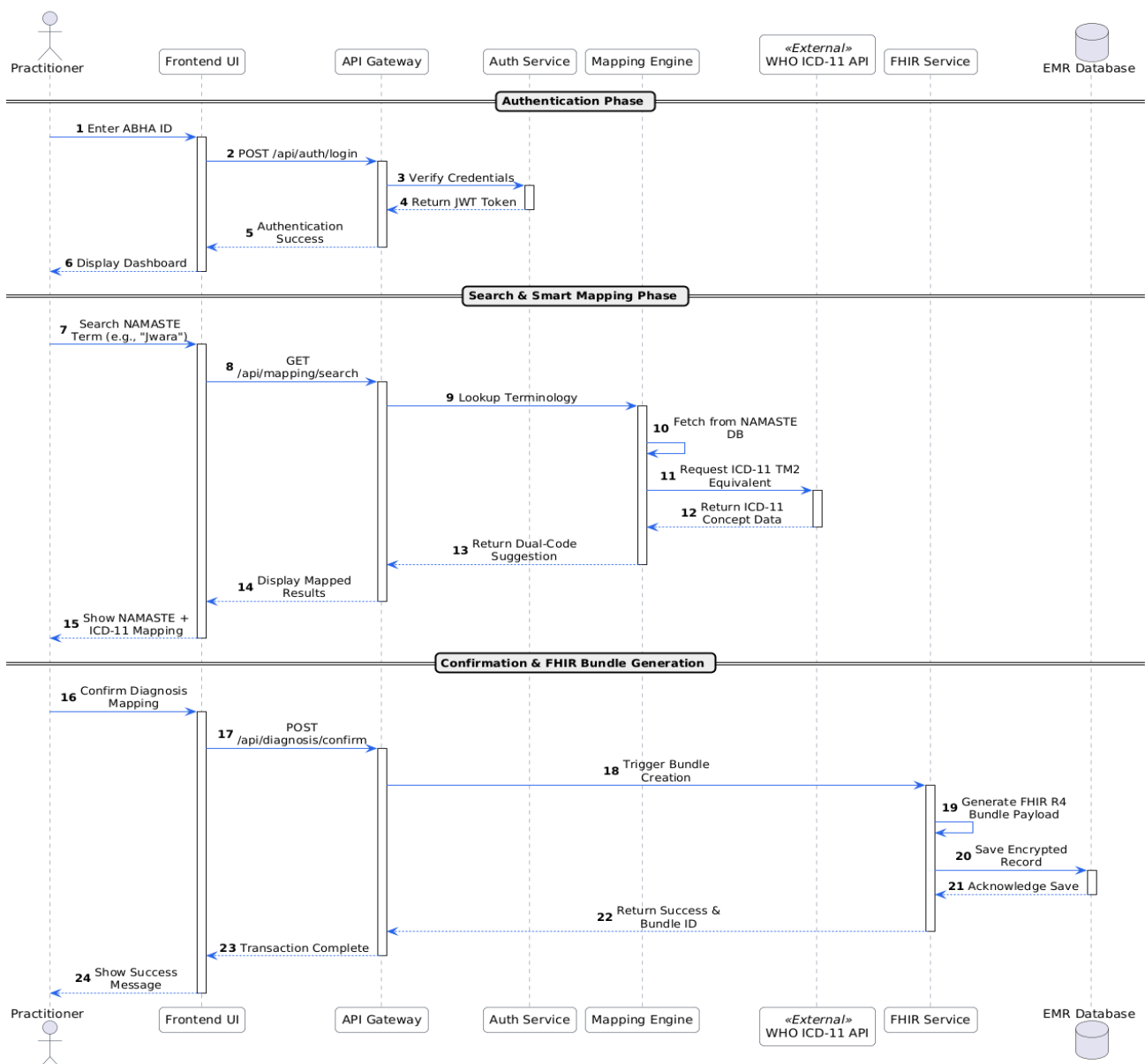
**Purpose:** Illustrates the step-by-step flow of messages and data between the system components over time during a diagnostic session.

**Key Elements:**

**Lifelines:** Represents the interacting entities: Practitioner, UI, API Gateway, core microservices (Auth, Mapping, FHIR), WHO API, and the EMR Database.

**Messages:** Arrow-directed communications showing REST API calls, data retrieval, and system responses.

**Description:** This diagram breaks down the dynamic workflow of the CodeVeda platform into three key phases. It starts with secure ABHA authentication, follows the process of searching a traditional NAMASTE term to fetch its ICD-11 TM2 equivalent from the WHO API, and concludes by generating a standardized FHIR bundle and saving it securely to the EMR database.



### Sequence Diagram

## 4. Activity Diagram

**Purpose:** Models the workflow and step-by-step business processes within the system.

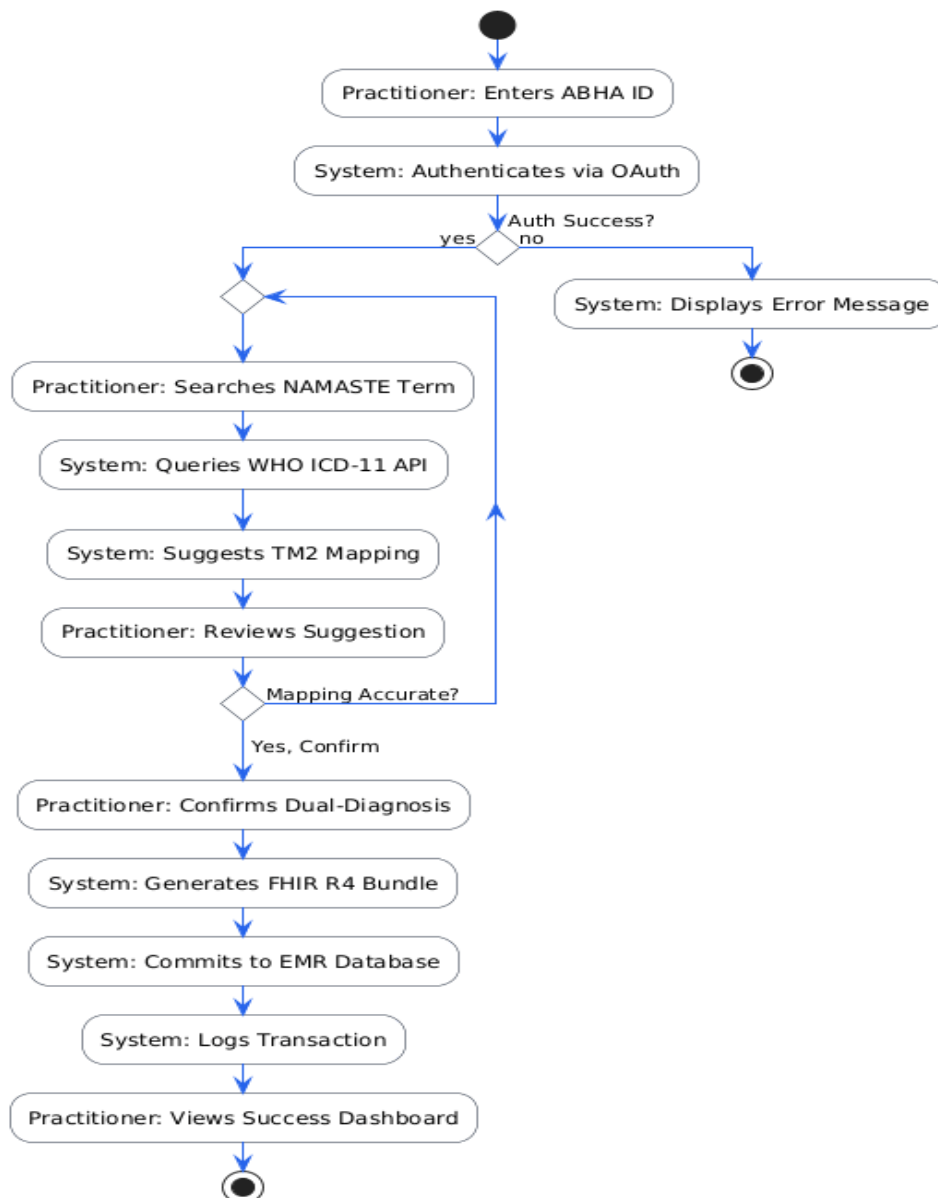
**Key Elements:**

**Activities:** Practitioner login, terminology search, API querying, mapping review, FHIR R4 bundle generation, database commit, and transaction logging.

**Decision Points:** Authentication validation ("Auth Success?") and mapping accuracy verification ("Mapping Accurate?").

**Start/End Points:** Process initiation at login, and termination at either a successful dashboard view or an error message.

**Description:** It highlights the logical path from secure ABHA authentication through an iterative search and mapping loop, culminating in the final generation of an FHIR bundle and secure EMR database commit



**Activity Diagram**

## 5. Object Diagram

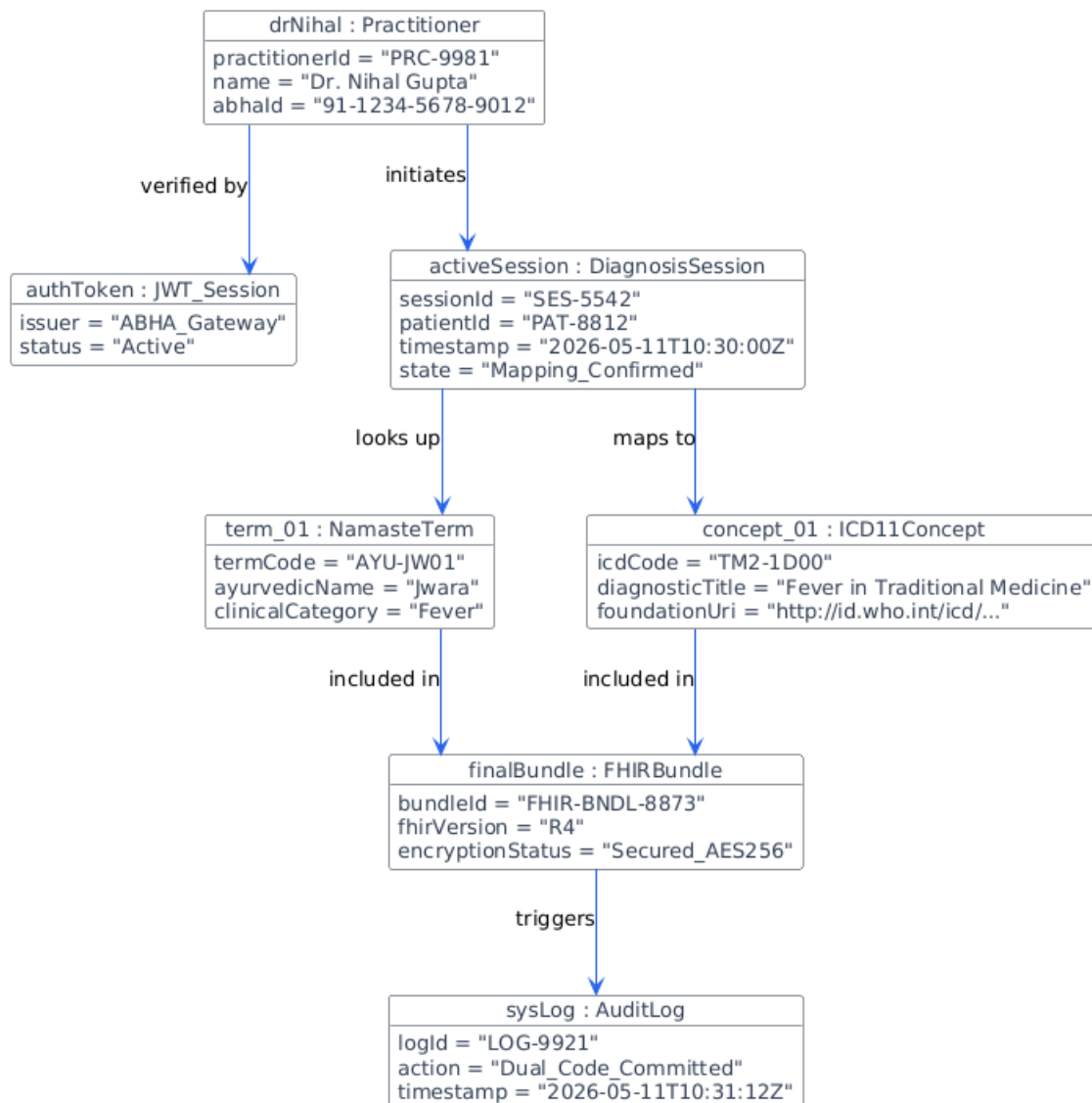
**Purpose:** Provides a real-time snapshot of the system's instantiated classes, their specific data values, and relationships during an active transaction.

### Key Elements:

**Objects:** Concrete instances of the system's architecture, including drNihal (Practitioner), authToken (JWT Session), activeSession (Diagnosis Session), specific terminology terms (term\_01, concept\_01), finalBundle (FHIR R4), and sysLog (Audit Logger).

**Attribute Values:** Captures actual runtime data, such as real terminology codes ("AYU-JW01" and "TM2-1D00"), active timestamps, and secure encryption statuses.

**Description:** It captures a specific moment where a verified practitioner is actively conducting a diagnosis. It visually demonstrates the "Dual-Coding" process by showing a traditional NAMASTE term and an ICD-11 concept merging to successfully instantiate a secured FHIR diagnostic bundle, which then immediately triggers a system audit log.



## Object Diagram

## 6. Component Diagram

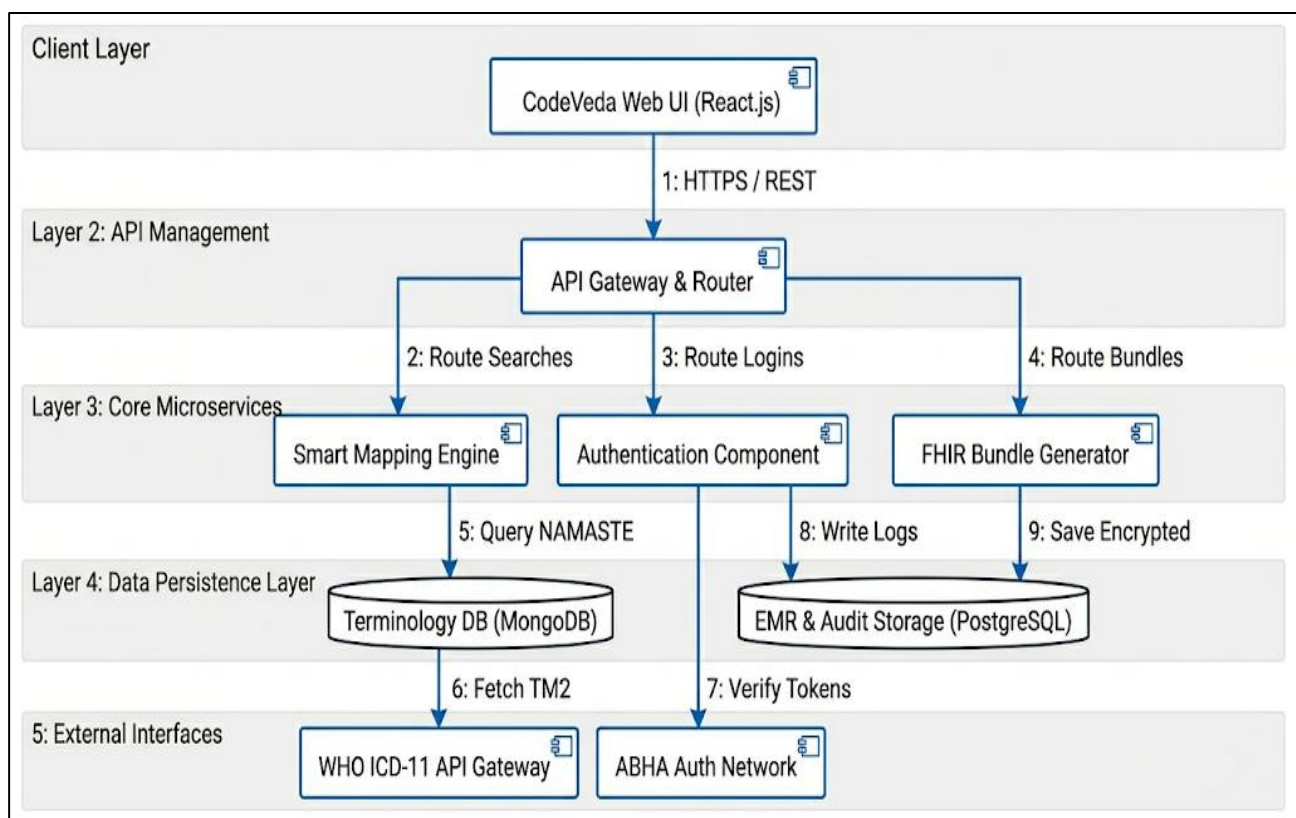
**Purpose:** Shows the structural relationships, routing, and dependencies between the software components, databases, and external APIs of the system.

### Key Elements:

**Components:** Core modules of the platform, including the CodeVeda Web UI (React.js), API Gateway, Authentication Component, Smart Mapping Engine, and FHIR Bundle Generator.

**Databases & Interfaces:** The Data Persistence Layer (MongoDB for terminology, PostgreSQL for EMR & Audit storage) and External Interfaces (WHO ICD-11 API and ABHA Auth Network).

**Description:** It provides a clean, high-level architectural view of the CodeVeda platform. It maps the vertical data flow, demonstrating how the API Gateway routes client requests to isolated microservices, which then securely interact with their respective persistence layers and external regulatory networks.



## Component Diagram



## 7. Deployment Diagram

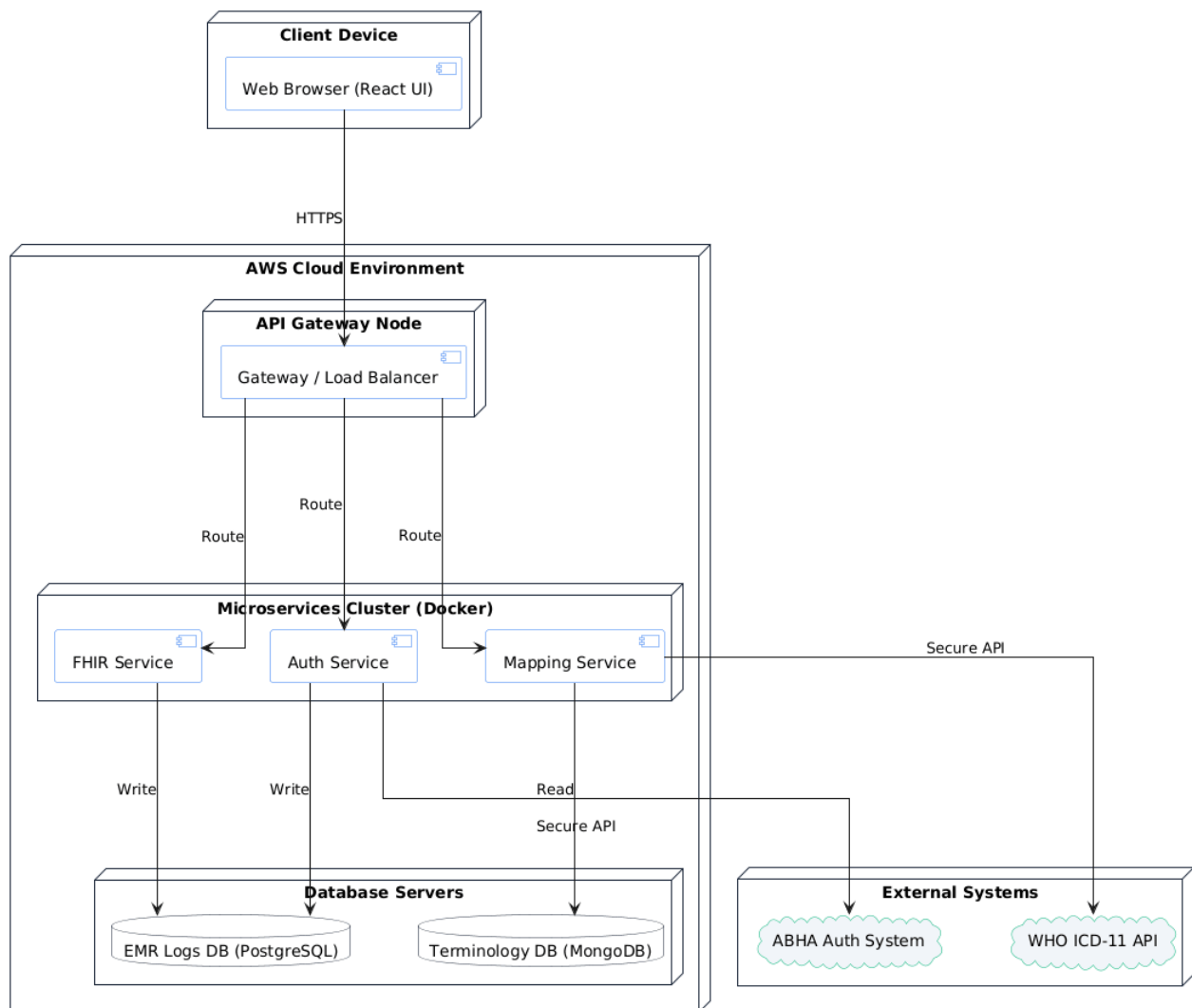
**Purpose:** Shows the physical deployment architecture and hardware/software relationships of the CodeVeda system.

**Key Elements:**

**Nodes:** Client Device, AWS Cloud Environment, and External Systems.

**Components:** Web Browser (React UI), API Gateway, Dockerized Microservices (FHIR, Auth, Mapping), and Database Servers (MongoDB, PostgreSQL).

**Description:** This diagram outlines the physical infrastructure of the platform. It demonstrates how the web client interacts with an AWS-hosted API Gateway, which routes requests to specific microservices. It also details the separation of the terminology and EMR databases, and highlights the secure HTTPS communication channels established with the external ABHA and WHO APIs.



**Deployment Diagram**

## 8. Communication Diagram

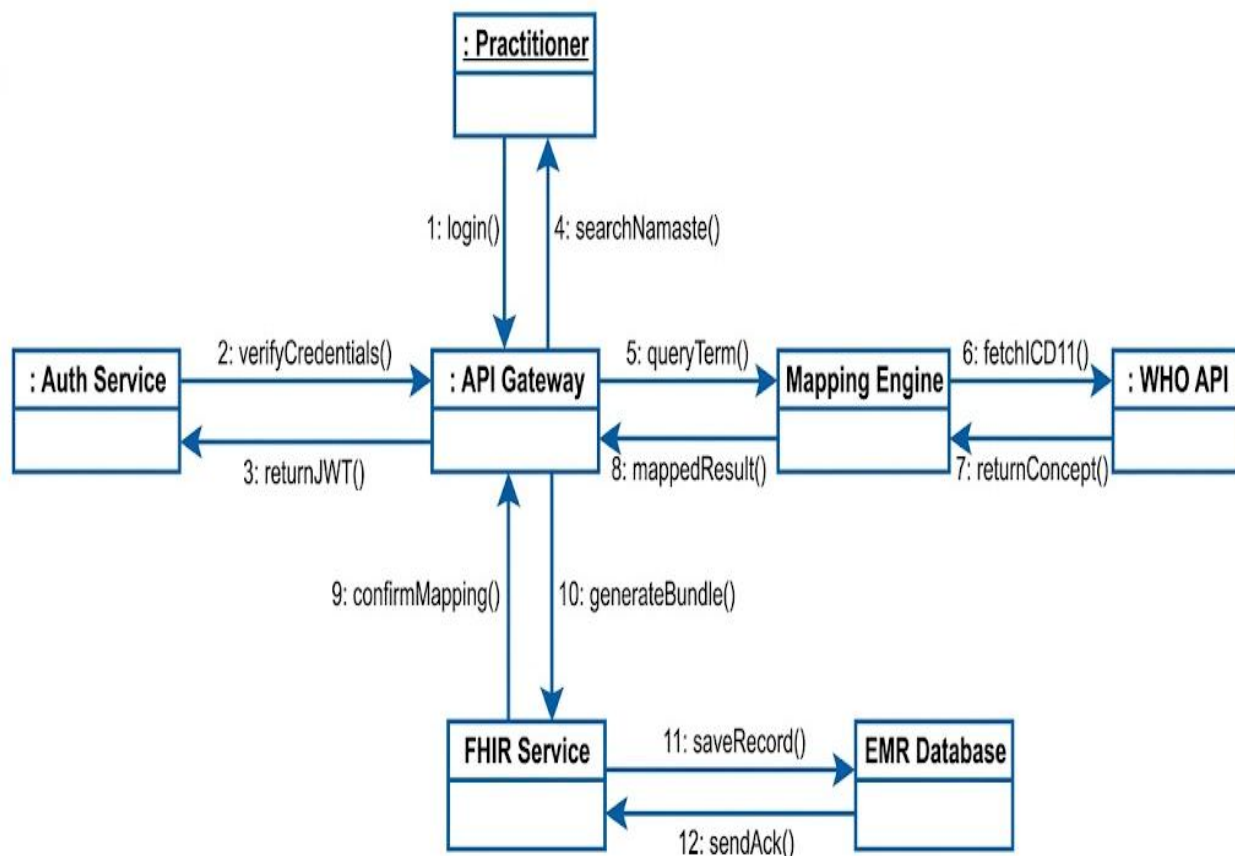
**Purpose:** Displays the structural organization of the system's objects and the exact sequential flow of messages passed between them to complete a diagnostic task.

**Key Elements:**

**Objects:** Key architectural nodes represented as blocks (Practitioner, API Gateway, Auth Service, Mapping Engine, WHO API, FHIR Service, and EMR Database).

**Numbered Messages:** Chronological method calls (labeled 1 to 12) showing the exact order of data requests and responses.

**Description:** This diagram provides a clear, network-style view of the CodeVeda backend. It highlights the central role of the API Gateway as the main orchestrator directing traffic. The numbered sequence tracks the entire diagnostic lifecycle: starting from practitioner authentication (Steps 1-3), moving through the terminology mapping process with external WHO APIs (Steps 4-8), and concluding with FHIR bundle generation and secure EMR database storage (Steps 9-12).



## Communication Diagram

## 9. Package Diagram

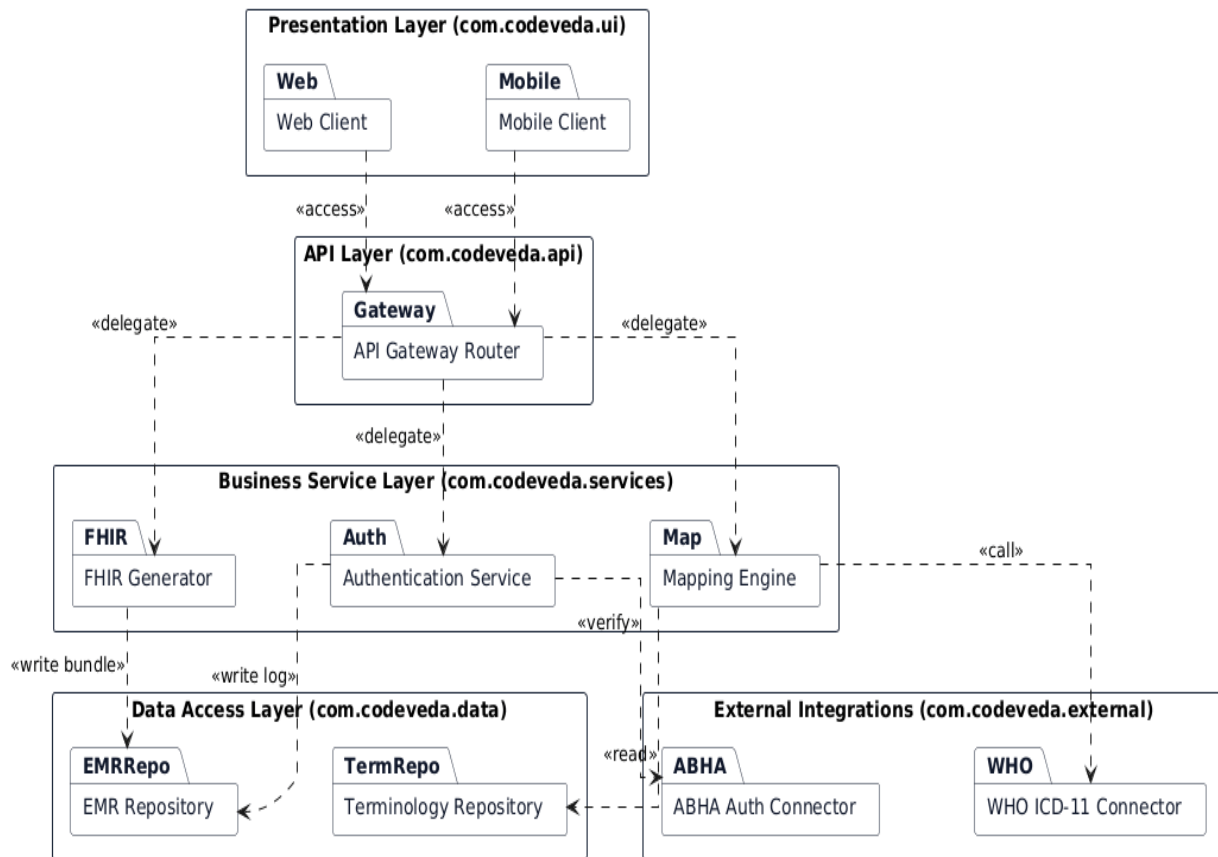
**Purpose:** Organizes the system's architecture into manageable, logical layers to provide a high-level view of modular dependencies.

**Key Elements:**

**Packages:** Distinct logical layers including the Presentation Layer, API Layer, Business Service Layer, Data Access Layer, and External Integrations.

**Dependencies:** Directed dashed arrows featuring standard UML stereotypes (like <<access>> and <<delegate>>) that show how higher-level packages rely on lower-level ones to function.

**Description:** This diagram represents the modular, layered architecture of the CodeVeda platform. It demonstrates a strong separation of concerns, showing how client applications interact with the API Gateway, which delegates complex tasks to underlying business services. These services then securely interface with distinct data repositories and external connectors (WHO and ABHA) without tangling the system's core logic.



## Package Diagram

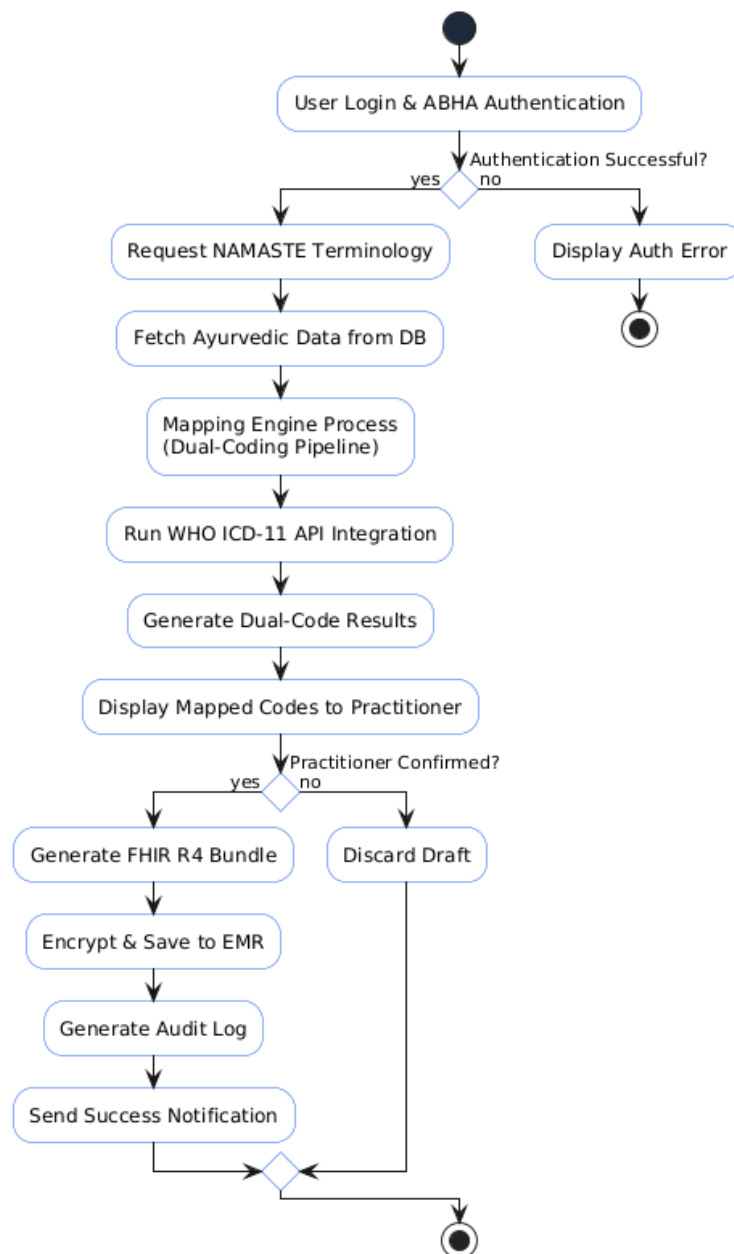
## 10. Interaction Diagram

**Purpose:** Illustrates the step-by-step operational workflow of the system, focusing on the decision logic and data processing paths.

**Key Elements: Action Nodes:** Core functional steps including ABHA Authentication, NAMASTE Fetching, Dual-Coding Pipeline, and FHIR R4 Bundle Generation.

**Control Flow:** Diamond-shaped decision points that handle conditional logic, such as authentication success/failure and practitioner confirmation of the mapped codes.

**Description:** It starts with a secure login, triggers a complex terminology mapping process involving external WHO APIs, and concludes with a secure, encrypted data commit. The flow ensures that clinical records are only saved once the dual-coding validation is complete and the practitioner has verified the results, maintaining high data integrity.



## Interaction Diagram

## **Conclusion**

The comprehensive set of UML diagrams presented in this report provides a complete specification of the **CodeVeda** platform from multiple perspectives:

**Functional View:** Use Case diagrams define core system goals, while Activity and Interaction flows ensure a seamless diagnostic mapping experience for practitioners

**Structural View:** Class, Component, and Package diagrams define the modular microservices architecture, ensuring high scalability and strict separation of clinical concerns.

**Behavioral View:** Sequence, Communication, and State Machine diagrams model the real-time logic of terminology dual-coding, specifically the transition from NAMASTE (Ayurvedic) to ICD-11 (Global) standards.

**Specialized Views:** Timing and Deployment diagrams provide domain-specific insights into system performance under HL7 FHIR R4 heavy-load conditions and secure cloud environments.

These diagrams serve as a definitive blueprint for system development, providing clear technical guidance for developers, ensuring regulatory compliance for ABHA (Ayushman Bharat Digital Mission) integration, and establishing a roadmap for future expansion into the global Electronic Health Record (EHR) market.

## **Technical Specifications**

**Development Stack:** MERN Stack (React.js, Node.js, Express.js)

**Healthcare Standards:** HL7 FHIR R4, WHO ICD-11, NAMASTE

**Data Management:** MongoDB Atlas (Terminology) & PostgreSQL RDS (Clinical Records)

**Security Framework:** ABHA OAuth 2.0, AES-256 Encryption, JWT Authentication

**Deployment:** AWS Healthcare Interoperability Hub, Vercel

**Institution:** Acropolis Institute of Technology and Research, Indore

### **Internal Guide:**

Prof. (Dr) Manish Vyas

### **External Guide:**

Mr. Lokesh Vishwakarma

### **Group Members:**

Naman Parmar (0827IT231089)

Nihal Gupta (0827IT231093)

Rishi Dubey (0827IT243D08)

Saksham Vishwakarma (0827IT243D09)